

OLAP Cube Analysis with MySQL

Introduction

This project demonstrates **OLAP (Online Analytical Processing)** operations using **MySQL and MySQL Workbench** on a coffee sales dataset.

The objective is to simulate multidimensional analysis by creating **fact tables, dimension tables, and pre-aggregated summary tables**, and then performing **slice, dice, and performance benchmarking** queries.

2. Tools & Technologies

- **Database:** MySQL 8.0
 - **Client Tool:** MySQL Workbench
 - **Dataset:** Coffee Sales Dataset (CSV)
 - **Concepts Used:**
 - OLAP
 - Star Schema
 - Fact & Dimension Tables
 - Aggregate Tables
 - Slice & Dice
 - Query Performance Comparison
-

3. Database Schema Design

Schema Name

olap_coffee_sales

Tables Created

- coffee_sales_raw – Raw transactional data
- fact_sales – Fact table with calculated measures
- dim_date – Date dimension
- dim_product – Product dimension
- dim_store – Store dimension
- **Aggregate tables:**
 - agg_sales_by_day
 - agg_sales_by_product
 - agg_sales_by_store

This structure follows a **star schema**, commonly used in OLAP systems.

4. Data Loading

- The cleaned CSV file was imported into MySQL using **LOAD DATA INFILE**.
 - Empty rows were removed and data types were handled properly.
 - The dataset contains **~100,000 transactions**.
-

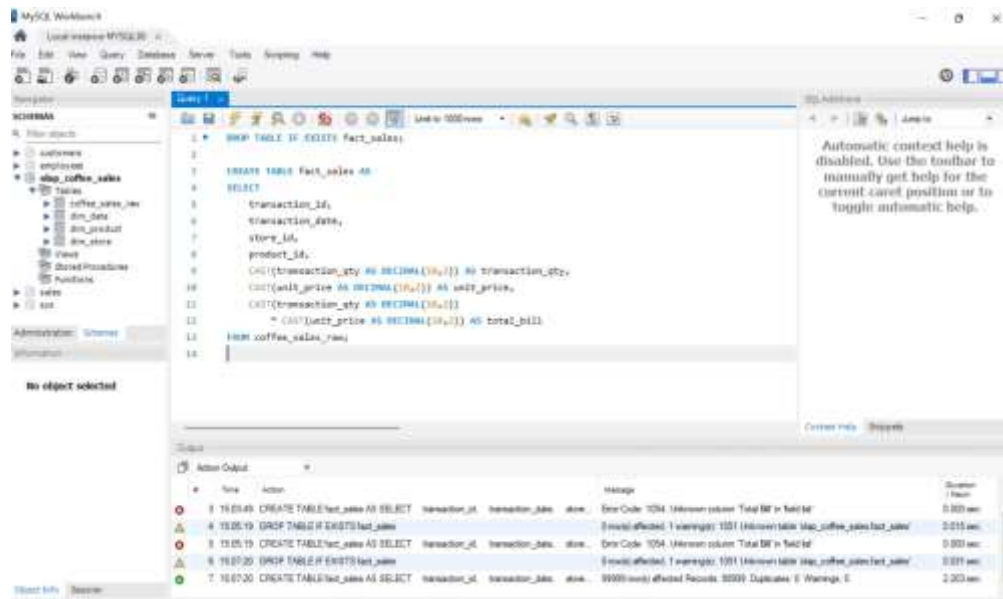
5. Fact Table Creation

fact_sales

The fact table stores numeric measures required for analysis.

Measures:

- transaction_qty
- unit_price
- total_bill (calculated as $\text{transaction_qty} * \text{unit_price}$)



6. Dimension Tables

6.1 Date Dimension (dim_date)

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'shop_coffee_sales' database selected. The 'Tables' pane shows 'dim_date' as the current table. The 'Columns' pane shows the columns: 'transaction_id', 'transaction_date', 'day_name', 'day_of_week', 'month_name', and 'month'. The 'Table' pane shows the table structure: 'transaction_id' (VARCHAR(10)), 'transaction_date' (DATE), 'day_name' (VARCHAR(10)), 'day_of_week' (TINYINT(4)), 'month_name' (VARCHAR(10)), and 'month' (TINYINT(4)). The 'Table' pane also shows the table is 'InnoDB' and 'UTF-8'.

The 'Table' pane shows the table structure:

transaction_id	transaction_date	day_name	day_of_week	month_name	month
1	2023-06-01	Thursday	4	June	6
2	2023-06-02	Friday	5	June	6
3	2023-06-03	Saturday	6	June	6
4	2023-06-04	Sunday	7	June	6
5	2023-06-05	Monday	1	June	6
6	2023-06-06	Tuesday	2	June	6
7	2023-06-07	Wednesday	3	June	6
8	2023-06-08	Thursday	4	June	6
9	2023-06-09	Friday	5	June	6

The 'Table' pane also shows the table is 'InnoDB' and 'UTF-8'.

6.2 Product Dimension (dim_product)

Stores product-related attributes.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'shop_coffee_sales' database selected. The 'Tables' pane shows 'dim_product' as the current table. The 'Columns' pane shows the columns: 'transaction_id', 'transaction_date', 'transaction_time', 'store_id', 'store_location', 'product_id', 'product_name', 'product_type', 'product_detail', 'size', and 'unit_price'. The 'Table' pane shows the table structure: 'transaction_id' (VARCHAR(10)), 'transaction_date' (DATE), 'transaction_time' (TIME), 'store_id' (TINYINT(4)), 'store_location' (VARCHAR(10)), 'product_id' (TINYINT(4)), 'product_name' (VARCHAR(10)), 'product_type' (TINYINT(4)), 'product_detail' (VARCHAR(10)), 'size' (TINYINT(4)), and 'unit_price' (TINYINT(4)). The 'Table' pane also shows the table is 'InnoDB' and 'UTF-8'.

The 'Table' pane shows the table structure:

transaction_id	transaction_date	transaction_time	store_id	store_location	product_id	product_name	product_type	product_detail	size	unit_price
1	2023-06-01	14:00:00	1	1	1	1	1	1	1	1
2	2023-06-02	14:00:00	1	1	1	1	1	1	1	1
3	2023-06-03	14:00:00	1	1	1	1	1	1	1	1
4	2023-06-04	14:00:00	1	1	1	1	1	1	1	1
5	2023-06-05	14:00:00	1	1	1	1	1	1	1	1
6	2023-06-06	14:00:00	1	1	1	1	1	1	1	1
7	2023-06-07	14:00:00	1	1	1	1	1	1	1	1
8	2023-06-08	14:00:00	1	1	1	1	1	1	1	1
9	2023-06-09	14:00:00	1	1	1	1	1	1	1	1

The 'Table' pane also shows the table is 'InnoDB' and 'UTF-8'.

6.3 Store Dimension (dim_store)

Stores store information.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'olap_coffee_sales' database selected. The 'Table: coffee_sales_raw' is highlighted. The 'Columns' pane lists the columns of the raw table. The main editor shows the SQL script for creating the 'dim_store' table. The 'SQL Assistant' pane on the right provides context help. The 'Action Output' pane at the bottom shows the execution results of the SQL script.

```
1 CREATE TABLE dim_store AS
2 SELECT DISTINCT
3   store_id,
4   store_location
5 FROM coffee_sales_raw;
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Time	Action	Message	Duration / Fetch
00 14:50:01	DROP TABLE IF EXISTS coffee_sales_raw	0 row(s) affected	0.047 sec
70 14:50:01	CREATE TABLE coffee_sales_raw (transaction_id VARCHAR(255), transaction_date VARCHAR(255), store_id VARCHAR(255), store_location VARCHAR(255), product_id VARCHAR(255), transaction_qty VARCHAR(255), unit_price VARCHAR(255), total_amt VARCHAR(255), product_category VARCHAR(255), product_type VARCHAR(255), product Retail Sale Month Name VARCHAR(255))	0 row(s) affected	0.047 sec
71 14:50:14	LOAD DATA INFILE C:\ProgramData\MySQL\MySQL Server 5.5\Uploads\coffee	30000 row(s) affected Records: 0 Deleted: 0 Skipped: 0 Warnings: 0	2.436 sec
72 14:50:20	DROP TABLE IF EXISTS coffee_sales_raw	0 row(s) affected	0.047 sec
73 14:50:20	CREATE TABLE dim_store AS SELECT DISTINCT product_id, product_cat	30 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.406 sec
74 14:50:20	CREATE TABLE dim_store AS SELECT DISTINCT store_id, store_location F	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.207 sec

7. Pre-Aggregated Tables (OLAP Cubes)

7.1 Sales by Day

The screenshot shows the MySQL Workbench interface. The main editor displays a SQL script for creating pre-aggregated tables. The 'SQL Assistant' pane on the right provides context help. The 'Action Output' pane at the bottom shows the execution results of the SQL script.

```
1 SELECT
2   transaction_date,
3   sum(total_amt) AS total_sales,
4   sum(transaction_qty) AS total_quantity
5 FROM fact_sales
6 GROUP BY transaction_date;
7
8 CREATE TABLE app_sales_by_day AS SELECT
9   transaction_date,
10  sum(total_amt) AS total_sales,
11  sum(transaction_qty) AS total_quantity
12 FROM fact_sales
13 GROUP BY transaction_date;
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Transaction Date	Store ID	Store Location	Total Sales	Total Quantity
2012-01-01	1	14th Avenue	1000.00	1000.00
2012-01-02	1	14th Avenue	1000.00	1000.00
2012-01-03	1	14th Avenue	1000.00	1000.00
2012-01-04	1	14th Avenue	1000.00	1000.00
2012-01-05	1	14th Avenue	1000.00	1000.00
2012-01-06	1	14th Avenue	1000.00	1000.00
2012-01-07	1	14th Avenue	1000.00	1000.00
2012-01-08	1	14th Avenue	1000.00	1000.00
2012-01-09	1	14th Avenue	1000.00	1000.00
2012-01-10	1	14th Avenue	1000.00	1000.00

7.2 Sales by Product

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL code:

```
1 SELECT
2     Product_ID,
3     SUM([total_sales]) AS total_sales,
4     SUM([transaction_qty]) AS total_quantity
5 FROM Fact_sales
6 GROUP BY Product_ID
7 ORDER BY total_sales DESC
```

The Results pane shows the following data:

Product_ID	total_sales	total_quantity
1	2210.0000	4094.00
2	2160.0000	3861.00
3	2000.0000	4002.00
4	2000.0000	3648.00
5	1970.0000	3942.00
6	1800.0000	4020.00
7	1710.0000	1710.00
8	1670.0000	1940.00

The Messages pane shows the following execution plan:

Step	Msg	Msg	Msg	Msg	Msg
1	21:43:47	SELECT FROM Fact_sales	0 rows returned		
2	21:43:47	GROUP BY Product_ID	0 rows affected		
3	21:43:47	ORDER BY total_sales DESC	0 rows affected		

7.3 Sales by Store

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL code:

```
1 SELECT
2     Store_ID,
3     SUM([total_sales]) AS total_sales,
4     SUM([transaction_qty]) AS total_quantity
5 FROM Fact_sales
6 GROUP BY Store_ID
7 ORDER BY total_sales DESC
```

The Results pane shows the following data:

Store_ID	total_sales	total_quantity
1	1800.0000	3648.00
2	1220.0000	3648.00
3	1800.0000	3648.00

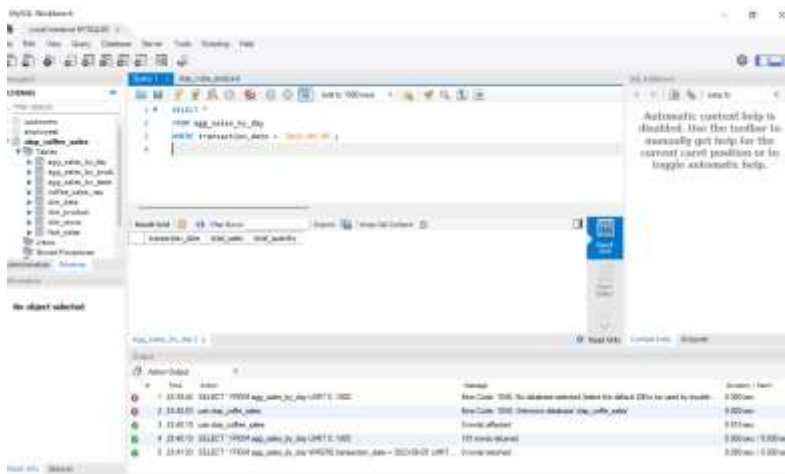
The Messages pane shows the following execution plan:

Step	Msg	Msg	Msg	Msg	Msg
1	21:43:47	SELECT FROM Fact_sales	0 rows returned		
2	21:43:47	GROUP BY Store_ID	0 rows affected		
3	21:43:47	ORDER BY total_sales DESC	0 rows affected		

8. OLAP Operations

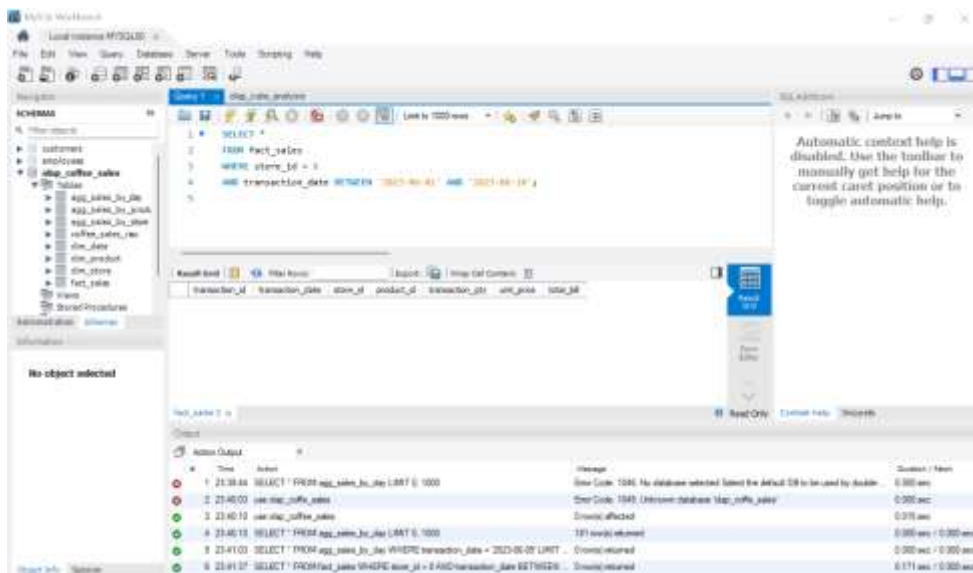
8.1 Slice

Sales for a specific day:

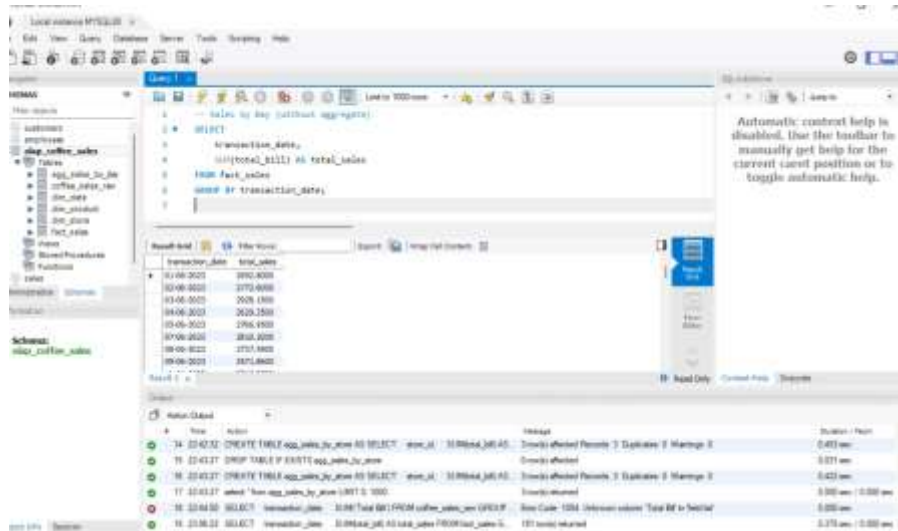


8.2 Dice

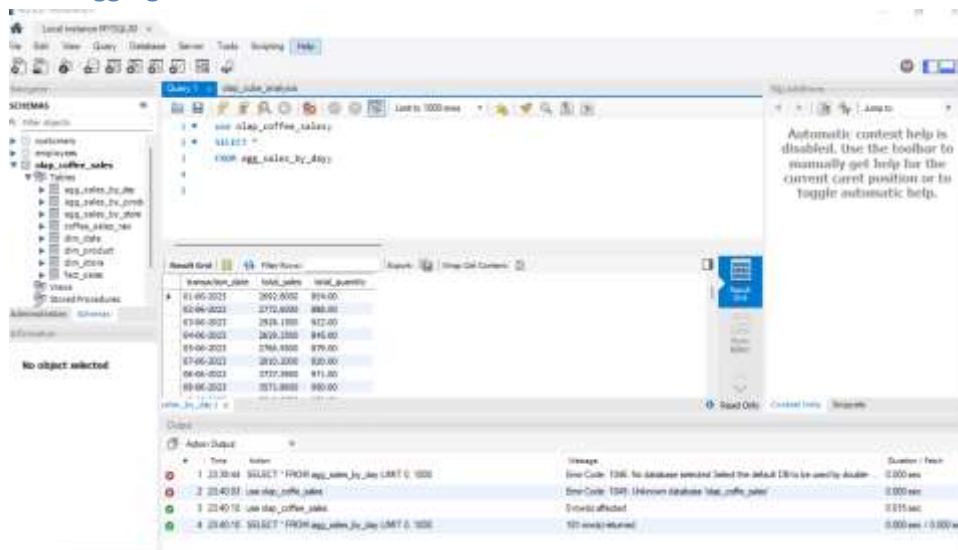
Sales for selected products and stores:



Without Aggregates



With Aggregates



Observation

- Queries on **aggregate tables executed significantly faster**
- Reduced computation overhead
- Demonstrates the benefit of OLAP pre-aggregation

10. Results & Insights

- Daily, product-wise, and store-wise sales trends were successfully analyzed

- Aggregate tables improved query performance
 - OLAP concepts were effectively simulated using MySQL
-

11. Conclusion

This project successfully demonstrates **OLAP cube analysis using MySQL** by:

- Designing a star schema
- Creating fact and dimension tables
- Implementing pre-aggregated summary tables
- Performing slice, dice, and benchmarking operations

The results clearly show how OLAP techniques enhance analytical query performance and support multidimensional decision-making.